World Wide Web (2025) 28:50
https://doi.org/10.1007/s11280-025-01358-8

Integrating context and criteria: a multi-head attention-based approach for multi-criteria group recommender systems

Ngoc Luyen Le[1] · **Marie-Hélène Abel**[1]

Received: 19 March 2025 / Revised: 20 May 2025 / Accepted: 12 June 2025 /
Published online: 11 July 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract
Group recommender systems (GRS) are essential for collective decision-making across various domains. However, traditional approaches often struggle to balance diverse group interests and tend to overlook crucial contextual factors and multi-criteria evaluations inherent in real-world choices. To address these limitations, we propose a Context-Aware Multi-Criteria Group Recommender System (CA-MCGRS) that employs a multi-head attention (MHA) mechanism. This architecture dynamically integrates individual user preferences, multiple evaluation criteria, and diverse contextual influences, enabling adaptive and nuanced recommendations. We conducted experiments on two distinct datasets: the ITM-Rec educational dataset, which includes multi-criteria ratings (e.g., application relevance, data quality, usability) and specific academic contexts (e.g., class, semester, lockdown status), and the widely-used MovieLens-100k dataset, adapted for group scenarios with simulated multi-criteria and contextual dimensions (e.g., time of day, season). Our results demonstrate that the MHA-based CA-MCGRS significantly improves predictive accuracy compared to various baseline models, particularly when leveraging rich multi-criteria information and adapting to specific contextual situations, such as the impactful lockdown status in the educational setting or temporal variations in the entertainment domain. Although the model exhibits longer training durations, it demonstrates superior generalization capabilities. These findings underscore the substantial benefits of integrating multi-criteria analysis and contextual awareness in GRS, offering valuable insights for developing more precise and relevant recommender systems.

Keywords Recommender system · Group recommender system · Multi-criteria decision making · Context-aware Multi-criteria recommender system

✉ Ngoc Luyen Le
ngoc-luyen.le@hds.utc.fr

✉ Marie-Hélène Abel
marie-helene.abel@hds.utc.fr

[1] Université de technologie de Compiègne, CNRS, Heudiasyc (Heuristics and Diagnosis of Complex Systems), Compiègne CS 60319 - 60203, France

1 Introduction

In today's digital age, recommender systems have become essential tools for providing personalized suggestions across various domains, including entertainment, retail, and education [18, 21, 43]. These systems analyze user preferences to suggest tailored items, such as movies, products, or educational resources, significantly enhancing user experience and decision-making efficiency. However, real-world decisions are frequently made collectively, introducing complexity as recommendations must cater to multiple, often diverse, individual preferences simultaneously. This shift from individual-focused systems to group-centric approaches presents substantial challenges, particularly in effectively aggregating and balancing differing user interests to ensure overall group satisfaction [33, 38].

Group Recommender Systems (GRS) aim to address these challenges by generating recommendations that reflect the collective preferences of a group rather than those of a single individual. While GRS has been successfully applied in domains such as tourism, e-learning, and e-commerce, their effectiveness is often hindered by the inherent difficulty of aggregating diverse and sometimes conflicting preferences among group members [6]. In traditional GRS, preference aggregation methods typically involve simple strategies such as averaging ratings or selecting items that maximize overall group satisfaction. However, these approaches fail to account for variations in individual priorities, leading to potential dissatisfaction for certain group members, particularly in heterogeneous groups with varying needs and preferences.

A limitation of many existing GRS is their reliance on single-criterion decision-making, which does not capture the complexity of real-world choices. In many decision scenarios, users evaluate items based on multiple criteria, such as quality, price, usability, or personal relevance. Different group members may prioritize different aspects of an item, making multi-criteria evaluation crucial for generating recommendations that align with the diverse expectations of the group. A system that only considers overall user preferences without examining specific criteria risks providing suboptimal recommendations, as it ignores the nuanced trade-offs individuals make in group decision-making [61].

Beyond multi-criteria considerations, another key factor in group recommendations is context. Contextual factors such as time, location, group composition, or the purpose of the recommendation significantly influence user preferences. For example, the same group of users may prefer different movie genres depending on whether they are watching on a weekend evening versus a weekday. Similarly, in an educational setting, student groups may have different project preferences based on the course structure, semester timeline, or external constraints such as a pandemic lockdown. Traditional GRS often overlook these contextual variations, reducing their ability to adapt recommendations to specific situations [2, 8].

To address these limitations, CA-MCGRS have emerged as a promising approach. These systems integrate three key dimensions of group recommendations: (1) collective group preferences, (2) multi-criteria decision-making, and (3) contextual awareness. By combining these elements, CA-MCGRS can offer more precise and context-sensitive recommendations that account for both individual preferences and external influences. Unlike traditional GRS, which primarily focus on preference aggregation, CA-MCGRS dynamically adapts recommendations based on situational factors, making them more applicable to real-world decision-making scenarios.

Building on our earlier conference work [30], this paper presents an extended and in-depth study of CA-MCGRS, a novel Context-Aware Multi-Criteria Group Recommender System. The model introduces a deep neural architecture based on Multi-Head Attention (MHA) that captures complex interactions between group members, contextual factors, and

evaluation criteria. Unlike traditional models that treat these dimensions separately, CA-MCGRS dynamically learns to prioritize the most relevant signals for each recommendation scenario. We validate our model on two datasets: ITM-Rec, which provides real-world educational group ratings under varying contexts, and MovieLens-100k, which we adapted to simulate multi-criteria and context-aware group recommendation settings. Our results show consistent performance gains over strong baselines across accuracy and ranking metrics, particularly in scenarios involving complex contextual or multi-criteria inputs. The main contributions of this paper are as follows: (i) We introduce CA-MCGRS, a deep learning architecture that unifies group preferences, criteria, and context via MHA; (ii) We propose a structured sequence modeling strategy for heterogeneous categorical inputs using stacked attention and cross-attention layers; (iii) We conduct experiments on two datasets (ITM-Rec and MovieLens-100k) across varying levels of contextual and criteria complexity; (iv) We evaluate performance using RMSE, MAE, NDCG@5, and Precision@5, and analyze training dynamics and computational cost; (v) We demonstrate that CA-MCGRS outperforms competitive deep learning baselines, particularly in context- and criteria-rich group scenarios.

The remainder of this paper is structured as follows: Section 2 reviews related research in the areas of context-aware and multi-criteria group recommender systems. Section 3 details the formulation of our problem and outlines the architecture of the proposed CA-MCGRS. Section 4 describes the experimental setup, including the datasets, baseline models, and evaluation metrics used. The empirical findings are then presented and analyzed in Section 5. We subsequently discuss the strengths, practical implications, and limitations of our approach in Section 6. Finally, Section 7 concludes the paper and provides directions for future research.

2 Related work

In this section, we discuss prior research on recommender systems, with a particular focus on GRS. We emphasize recent advances in context-awareness, multi-criteria decision-making, and their integration to address diverse group recommendation scenarios.

2.1 Recent advances in group recommender systems

Recommender Systems (RSs) have profoundly transformed how users interact with digital content, effectively navigating vast information landscapes by delivering personalized suggestions based on individual preferences [20, 42]. While traditional RSs primarily focus on individual user preferences [19], the growing prevalence of group-oriented activities – such as selecting travel destinations, educational projects, or entertainment – has elevated the importance of GRSs [33, 38].

GRSs aggregate individual user preferences to generate recommendations that balance the varied interests within a group [6]. Recent studies have explored numerous strategies to effectively handle group recommendation challenges, such as preference aggregation methods, fairness, and conflict resolution [23, 55]. Traditional aggregation strategies typically fall into two categories: (i) profile aggregation methods that form unified group profiles [16], and (ii) personalized recommendation aggregation, which individually recommends items before aggregating them into a unified group suggestion [38]. Emerging hybrid strategies combine the strengths of both approaches, demonstrating improved accuracy and fairness [17]. Furthermore, recent advancements include leveraging graph neural networks (GNNs)

to model complex intra-group interactions and user-item-group relationships [15, 25], and exploring reinforcement learning for dynamic group preference modeling and sequential recommendations [29].

Despite significant progress, GRSs continue to struggle with effectively integrating multiple evaluation criteria and contextual information, limiting the relevance and personalization of group recommendations in complex decision-making scenarios.

2.2 Context-awareness in group recommender systems

Context-aware recommender systems (CARS) improve traditional models by incorporating context – conditions or circumstances affecting user decisions – into the recommendation process [2]. Contextual factors, such as time, location, or social environment, significantly influence user preferences and choices. Recent studies emphasize that explicitly modeling such factors can substantially enhance recommendation quality and user satisfaction [1, 26, 34].

Current approaches include pre-filtering, post-filtering, and context-aware factorization models, which efficiently adapt recommendations to evolving contexts [34]. Recent deep learning advancements, such as contextual embeddings and attention mechanisms, further improve the dynamic integration of context, resulting in more accurate and context-sensitive recommendations [31, 58].

Several studies have advanced CARS by employing deep learning and knowledge-based techniques. For instance, one approach integrates bidirectional long short term memory with hybrid attention mechanisms to capture complex dependencies among users, items, and contextual factors [36]. In the tourism domain, ARTEMIS incorporates both user context and predicted crowd levels to improve recommendation relevance and user experience [35]. Another line of work leverages knowledge graphs, combining embeddings with convolutional networks to better model user–item relationships and enhance personalized recommendations [49].

However, most existing CARS approaches focus on individual recommendation scenarios. Limited attention has been given to group-based recommendations, leaving a notable research gap in understanding how contextual factors interact with the heterogeneous preferences of group members.

2.3 Multi-criteria recommender systems

Multi-Criteria Recommender Systems (MCRS) extend traditional recommender systems by evaluating items based on multiple attributes or dimensions, resulting in more nuanced and personalized recommendations [3, 32, 45]. To capture diverse user preferences, MCRS commonly employ methods such as multi-attribute utility theory, weighted sum techniques, and multi-objective optimization [12].

Recent research highlights that incorporating multiple criteria can significantly enhance both recommendation accuracy and user satisfaction [4]. For example, Salau et al. [44] proposes a hybrid DeepFM-SVD++ model that integrates deep learning with factorization techniques to effectively model multi-criteria interactions. Their experiments demonstrate superior performance over traditional methods, particularly in handling data sparsity and achieving cross-domain generalization. Similarly, Rajput et al. [41] introduces an adaptive attention-based approach that dynamically assigns weights to each criterion according to user preferences. Combined with a nearest-neighbor strategy, this model achieves higher

precision and lower error rates on the Yahoo! Movies dataset, outperforming both single- and multi-criteria baselines.

Applying multi-criteria approaches to group settings, however, introduces additional complexity. Aggregating diverse criteria preferences across group members necessitates advanced methods such as fuzzy aggregation, neural network-based aggregations, or adaptive weighting schemes [8, 51]. Despite these efforts, effectively integrating multi-criteria methods into GRS remains a relatively less explored area that requires deeper investigation to handle group heterogeneity more effectively.

2.4 Integrating context and multi-criteria in group recommender systems

Integrating context-awareness and multi-criteria evaluations within GRS has emerged as a compelling research direction, promising substantial improvements in recommendation relevance and satisfaction [8, 51]. By jointly modeling context and multiple criteria, systems can better understand user preferences and offer more targeted recommendations adapted to specific situations.

Recent studies employing attention-based deep learning models have shown promising results in capturing complex feature interactions in recommendations [37]. MHA mechanisms, initially popularized by transformer architectures, have demonstrated particular effectiveness in dynamically weighting criteria and contextual features based on their relevance, significantly enhancing recommendation accuracy and interpretability [31, 36].

Despite these advancements, a limited number of existing studies comprehensively address the integration of context-awareness, multi-criteria evaluation, and group dynamics simultaneously. Filling this research gap can lead to more effective, personalized, and contextually relevant group recommender systems.

To address these limitations, this paper presents a novel CA-MCGRS, leveraging a MHA mechanism to dynamically integrate contexts and criteria, significantly enhancing group recommendation performance. In the following section, we detail our proposed CA-MCGRS, including its task formulation and neural architecture designed to seamlessly integrate contextual and multi-criteria information tailored specifically for group scenarios.

3 Developing context-aware multi-criteria group recommender systems based on multi-head attention

In this section, we formalize the problem of designing a CA-MCGRS that incorporates multiple criteria, contextual information, and group dynamics to predict the overall group satisfaction for non-interacted items. We then present our deep neural network architecture for addressing this complex task.

3.1 Task formulation

The objective of developing a CA-MCGRS is to accurately predict and recommend a list of relevant items to groups by integrating contextual factors, multiple evaluation criteria, and considering group size. Therefore, we need to ensure that the recommendations are tailored to collective preferences, contextual influences, and various criteria.

Given that users $U = \{u_1, u_2, \ldots, u_n\}$ represents the set of individual users; groups $G = \{g_1, g_2, \ldots, g_k\}$ denotes the set of groups, where each group $g \in G$ is a subset of U,

i.e., $g \subseteq U$. Each group g has a size $|g|$, representing the number of members; items $I = \{i_1, i_2, ..., i_m\}$ represents the set of items to be recommended; contexts $C = \{c_1, c_2, ..., c_p\}$ represents the set of contexts affecting interactions with items, $CR = \{cr_1, cr_2, ..., cr_o\}$ represents the set of criteria through ratings, and $R = \{r_1, r_2, ..., r_q\}$ represents the set of overall ratings. The task recommendation of the CA-MCGRS can be defined as follows:

$$f(r) : G \times I \times C \times CR \longrightarrow R \tag{1}$$

where $G \times I \times C \times CR$ represents the Cartesian product of groups, items, contexts, and criteria, covering all possible group-item interactions influenced by different contextual factors and criteria. Each group $g \in G$ is a collection of several users. CR encompasses multiple criteria ratings, providing an evaluation of items depending on various aspects. Each criteria rating cr is given by group g to item i under context c. The overall rating R is derived from the evaluation of the group for an item under a given context. This overall rating value serves as the primary metric for generating recommendations in the CA-MCGRS.

For each group $g \in G$, the goal of the CA-MCGRS is to recommend a ranked list of the top-K items $I^g_{topK} \subseteq I^g_{ni}$ that are most likely to receive high overall ratings $R(g, i, c)$. This ranking process should take into account the following factors: (i) Contextual Factors: the contextual elements that influence how the group interacts with the items. (ii) Multi-Criteria Evaluations: assessments based on multiple criteria that evaluate different aspects of each item, as illustrated in Table 1.

The primary objective is to maximize the overall ratings of the recommended items. This optimization goal can be mathematically expressed as:

$$\max_{I^g_{topK}} \sum_{i \in I^g_{topK}} r(g, i, c, cr) \tag{2}$$

where item I^g_{ni} denotes the set of items that group g has not yet interacted with. And $r(g, i, c, cr)$ is the overall rating assigned to item i by group g under context c based on criteria cr.

To illustrate how recommendations are made for groups within a specific context and across multiple criteria, consider the following scenario where a university class is selecting projects for different student groups:

- **Students**: $U = \{u_1, u_2, u_3, u_4, u_5, u_6, u_7, u_8, u_9, u_{10}, u_{11}, u_{12}\}$
- **Groups**: $g_1 = \{u_1, u_2, u_3\}$, ($|g_1| = 3$); $g_2 = \{u_4, u_5\}$, ($|g_2| = 2$); $g_3 = \{u_6, u_7, u_8, u_9\}$, ($|g_3| = 4$); $g_4 = \{u_{10}, u_{11}, u_{12}\}$, ($|g_4| = 3$)
- **Projects**: $I = \{$*File Management System*, *Question Answering System*, *Mushroom Classification*, *Zika Virus Epidemic*$\}$
- **Contexts**: $C = \{$Class (c_1), Semester (c_2), Lockdown (c_3)$\}$

Table 1 Groups' evaluation of projects under various contexts and criteria

G	I	C			CR			R
		c_1	c_2	c_3	cr_1	cr_2	cr_3	
g_1	File Management System	DM	Spring	POS	5	5	4	5
g_2	Question Answering system	DA	Fall	POS	4	4	4	3
g_3	Mushroom Classification	DB	Spring	PRE	3	5	4	3
g_4	Zika Virus Epidemic	DM	Spring	PRE	2	4	5	5

- **Criteria**: $CR = \{\text{Applicability } (cr_1), \text{Data Quality } (cr_2), \text{Ease of Use } (cr_3)\}$

In this scenario, the RS needs to account for group size and composition when aggregating preferences. It must also integrate contextual and multi-criteria evaluations to recommend projects that align with the group's collective preferences. For example, recommending the "*File Management System*" project to g_1 (with group size: 3) involves considering how the group's preferences, contextual factors, and criteria ratings predict the best project match for that group.

The task formulation for a CA-MCGRS involves defining the sets of users, groups (including their sizes), items, contexts, and criteria, as well as establishing a function that maps these elements to overall ratings. The primary objective is to predict and recommend the top-K items to each group by leveraging multi-criteria ratings and contextual information. In the following section, we will delve into our deep neural network architecture, which is designed to effectively perform such recommendations.

3.2 Our deep neural network architecture for CA-MCGRS leveraging MHA mechanism

The CA-MCGRS architecture (Context-Aware Multi-Criteria Group Recommender System) is designed to generate personalized group-based recommendations by integrating group preferences with contextual information and evaluation criteria. Its core innovation lies in its ability to dynamically identify and prioritize the most relevant features for each recommendation scenario using an attention-based mechanism.

The architecture is composed of several modular components that collectively learn rich, context-sensitive representations. These include embedding layers for categorical features, positional encoding to preserve feature order, multi-head attention modules for dynamic relevance modeling, a cross-interaction network to capture higher-order feature relationships, and fully connected layers for final prediction. The complete workflow of the architecture is illustrated in Figure 1. To clarify how the model operates, consider a toy example: a group of three users (u_1, u_2, and u_3) is evaluating a project titled "Mushroom Classification" during the spring semester. The group provides feedback based on multiple evaluation criteria such as applicability, data quality, and ease of use. Each of these categorical inputs – group identity, item, context, and criteria – is transformed into a numerical embedding vector via trainable embedding layers. These embeddings are combined into a sequence, which also includes a special classification token (`[CLS]`) used to aggregate global information. Unlike transformer models such as BERT that process natural language sequences [7], CA-MCGRS processes sequences of structured categorical fields. The MHA mechanism enables the model to attend selectively to important inputs and interactions depending on the recommendation context. This attention-based design allows the model to flexibly capture how group composition, item characteristics, contextual conditions, and evaluation criteria interact in each situation. Rather than relying on fixed feature combinations, CA-MCGRS acts as a dynamic feature selector, adapting its focus to the most relevant signals for each group recommendation task.

3.2.1 Input and embedding layer

The input to the model consists of four structured components: group preferences G, item attributes I, contextual information C, and evaluation criteria CR. Each of these components contains categorical fields, which are transformed into low-dimensional continuous vectors using trainable embedding functions.

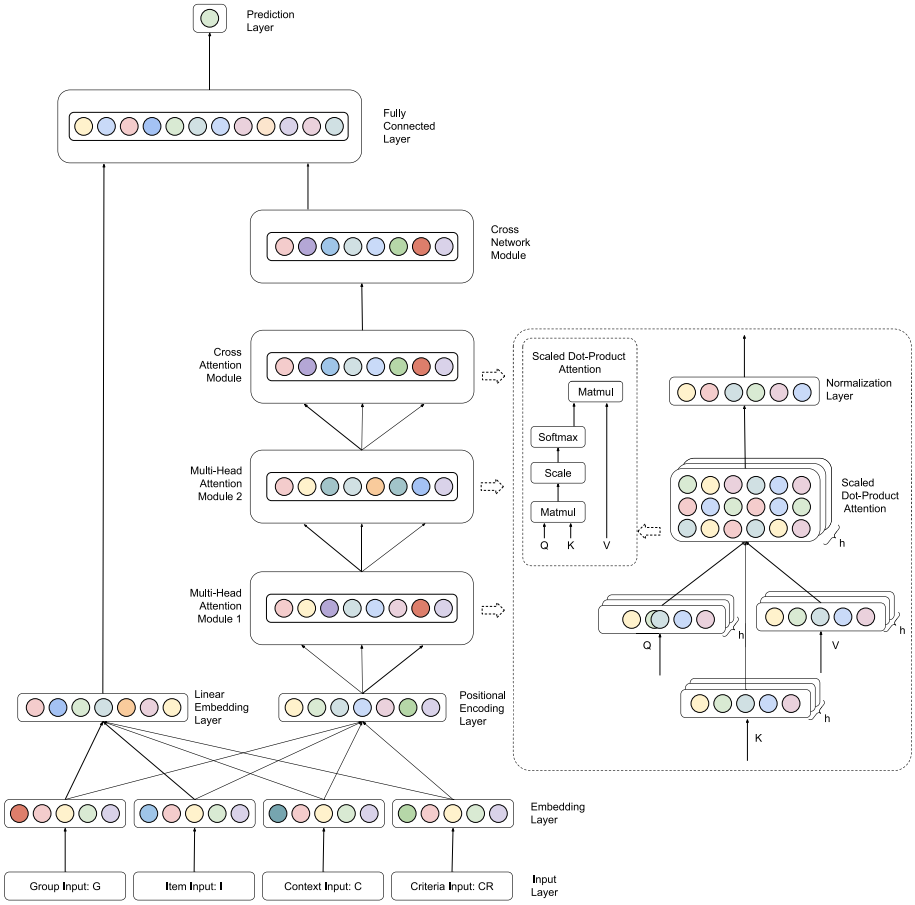


Fig. 1 The deep neural network architecture for the CA-MCGRS using MHA Mechanism

Let the embedded representations be defined as:

$$\begin{aligned} E_G &= \text{Embedding}(G) \\ E_I &= \text{Embedding}(I) \\ E_C &= \text{Embedding}(C) \\ E_{CR} &= \text{Embedding}(CR) \end{aligned} \tag{3}$$

where $E_G, E_I, E_C, E_{CR} \in \mathbb{R}^{n \times d}$ are the resulting embedding matrices for each input component. d is the embedding dimension.

3.2.2 Linear embedding layer

The linear embedding layer captures first-order feature contributions by applying a weighted linear combination over the same embedding vectors produced in the input embedding layer. These vectors include all field-wise embeddings from the group, item, context, and criteria

components, i.e., E_G, E_I, E_C, E_{CR}. Let $\{E_j\}_{j=1}^{n}$ denote the set of all individual feature embeddings extracted from these components, where each $E_j \in \mathbb{R}^d$ corresponds to one categorical input field.

Conventionally, the linear output is computed as:

$$Z_{\text{linear}} = \sum_{j=1}^{n} W_j^{\top} E_j + b_{\text{linear}} \in \mathbb{R} \tag{4}$$

where $W_j \in \mathbb{R}^d$ is the learnable weight vector associated with the j-th feature, and $b_{\text{linear}} \in \mathbb{R}$ is a scalar bias term, n is the number of fields.

This formulation enables the model to linearly aggregate information across all embedded input fields – spanning group, item, context, and criteria – without modeling their higher-order interactions.

3.2.3 Positional encoding layer

Transformer-based architectures operate entirely on attention mechanisms and are inherently permutation-invariant; they lack any built-in notion of sequence order. To compensate for this, a mechanism called *positional encoding* is introduced to provide the model with information about the relative or absolute position of each element in the input sequence.

Let $E = [\texttt{[CLS]}, E_G, E_I, E_C, E_{CR}] \in \mathbb{R}^{(n+1)\times d}$ denote the sequence of embedding vectors, where each $E_x \in \mathbb{R}^d$ represents a specific input field. The first vector, `[CLS]`, is a learnable classification token prepended to the sequence and used to capture global information aggregated across all input fields.

In our architecture, we adopt a learnable positional encoding matrix $P \in \mathbb{R}^{(n+1)\times d}$, where $n + 1$ corresponds to the total number of input fields, including the `[CLS]` token, and d is the embedding dimension. Each row $P_i \in \mathbb{R}^d$ of the matrix provides a distinct positional embedding for the i-th token in the input.

The positional encoding is combined with the input embeddings through element-wise addition:

$$E_{\text{pos}} = E + P$$

This produces a position-aware representation $E_{\text{pos}} \in \mathbb{R}^{(n+1)\times d}$, which retains both the semantic information of the input fields and their positional context. The resulting representation is passed to the multi-head attention module, allowing the model to attend over the structured input sequence while taking both content and position into account.

3.2.4 Multi-head attention modules

To model high-order dependencies between group-level inputs, items, contextual factors, and evaluation criteria, the architecture incorporates stacked MHA modules. This mechanism, introduced by Vaswani et al. [50], enables the model to attend to different representation subspaces simultaneously, thus capturing diverse interaction patterns between input fields.

The input to the attention modules is the positionally enriched embedding sequence $E_{\text{pos}} \in \mathbb{R}^{(n+1)\times d}$, where n is the number of embedded input fields, and the extra token corresponds to the learnable `[CLS]` embedding that serves as a global aggregator.

First Multi-Head Attention Module:
In the MHA mechanism, a head refers to an individual self-contained attention mechanism that processes the same input sequence but in a different learned projection subspace. Particularly, each attention head projects the input into three spaces: *Query* (Q), *Key* (K), and *Value* (V).

Given the input $E_{\text{pos}} \in \mathbb{R}^{(n+1)\times d}$, a single attention head i projects it into $Q^{(1)}$, $K^{(1)}$, and $V^{(1)}$ as follows:

$$Q_i^{(1)} = E_{\text{pos}} W_Q^{(1,i)}, \quad K_i^{(1)} = E_{\text{pos}} W_K^{(1,i)}, \quad V_i^{(1)} = E_{\text{pos}} W_V^{(1,i)} \tag{5}$$

where $W_Q^{(1,i)}, W_K^{(1,i)}, W_V^{(1,i)} \in \mathbb{R}^{d\times d_h}$ are trainable projection matrices.

For each head $i \in \{1, \ldots, h\}$, the attention output is computed as:

$$\text{head}_i^{(1)} = \text{Attention}(Q_i^{(1)}, K_i^{(1)}, V_i^{(1)}) = \text{Softmax}\left(\frac{Q_i^{(1)}(K_i^{(1)})^\top}{\sqrt{d_h}}\right) V_i^{(1)} \tag{6}$$

Each head operates in its own projection subspace of dimensionality $d_h = d/h$. The scaling factor $\sqrt{d_h}$ is used to stabilize gradients during training by counteracting large dot-product magnitudes.

The output from each attention head captures feature interactions in a distinct learned subspace. These outputs are concatenated along the feature dimension to form a single matrix, which is then projected back to the original embedding space using a shared linear transformation. Formally, the MHA output $\mathcal{A}$ is given by:

$$\mathcal{A}(E_{\text{pos}}) = \left[\text{head}_1^{(1)}, \ldots, \text{head}_h^{(1)}\right] W_O^{(1)} \tag{7}$$

where $W_O^{(1)} \in \mathbb{R}^{hd_h\times d}$ is a learnable projection matrix that maps the concatenated output of all h heads (each of size d_h) back to the model's embedding dimension d. Thus, $\mathcal{A}(\cdot)$ represents the complete MHA function, aggregating diverse attention patterns across heads into a unified, context-enriched representation of the input sequence.

The output of the first MHA module is passed through *Layer Normalization* [22] to stabilize training and reduce internal covariate shift. This operation normalizes each token's embedding across feature dimensions. Given a vector $\mathbf{x} \in \mathbb{R}^d$, the *Layer Normalization* $\mathcal{LN}(\cdot)$ is defined as:

$$\mathcal{LN}(\mathbf{x}) = \gamma \cdot \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta \tag{8}$$

with

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

where $\gamma, \beta \in \mathbb{R}^d$ are learnable parameters, and ϵ is a small constant for numerical stability.

Formally, the output of the first MHA module is:

$$Z^{(1)} = \mathcal{LN}(E_{\text{pos}} + \mathcal{A}(E_{\text{pos}})) \tag{9}$$

where $\mathcal{A}(E_{\text{pos}}) \in \mathbb{R}^{(n+1)\times d}$ denotes the output of the MHA transformation, and $\mathcal{LN}(\cdot)$ represents the layer normalization function. This step ensures that the normalized sequence $Z^{(1)}$ maintains zero mean and unit variance across feature dimensions, thereby improving numerical stability and enabling more effective learning in subsequent attention layers.

Second Multi-Head Attention Module:
The normalized output from the first MHA module, $Z^{(1)} \in \mathbb{R}^{(n+1)\times d}$, serves as the input to the second MHA module. This second MHA module performs the same sequence of operations: linear projections into query, key, and value subspaces, computation of scaled dot-product attention, concatenation of heads, and final output projection.

Formally, for each head $i \in \{1, \ldots, h\}$, the attention operations are defined as:

$$Q_i^{(2)} = Z^{(1)} W_Q^{(2,i)}, \quad K_i^{(2)} = Z^{(1)} W_K^{(2,i)}, \quad V_i^{(2)} = Z^{(1)} W_V^{(2,i)} \tag{10}$$

$$\text{head}_i^{(2)} = \text{Softmax}\left(\frac{Q_i^{(2)} (K_i^{(2)})^\top}{\sqrt{d_h}}\right) V_i^{(2)} \tag{11}$$

$$\mathcal{A}(Z^{(1)}) = \left[\text{head}_1^{(2)}, \ldots, \text{head}_h^{(2)}\right] W_O^{(2)} \tag{12}$$

where $Z^{(1)} \in \mathbb{R}^{(n+1)\times d}$ denotes the input to the second MHA module. For each head $i \in \{1, \ldots, h\}$, the query, key, and value matrices $Q_i^{(2)}, K_i^{(2)}, V_i^{(2)} \in \mathbb{R}^{(n+1)\times d_h}$ are obtained by linearly projecting the shared input $Z^{(1)}$ using learned weight matrices $W_Q^{(2,i)}, W_K^{(2,i)}, W_V^{(2,i)} \in \mathbb{R}^{d\times d_h}$. Each head computes a weighted representation $\text{head}_i^{(2)} \in \mathbb{R}^{(n+1)\times d_h}$ through scaled dot-product attention. The outputs of all heads are then concatenated and projected back to the original embedding dimension using a learned output matrix $W_O^{(2)} \in \mathbb{R}^{hd_h\times d}$, producing the final attention output $\mathcal{A}(Z^{(1)}) \in \mathbb{R}^{(n+1)\times d}$. The output of the second attention module is again combined with its input through a residual connection and normalized:

$$Z^{(2)} = \mathcal{LN}(Z^{(1)} + \mathcal{A}(Z^{(1)})) \tag{13}$$

The second residual-attention block allows the model to further refine the representations learned in the first layer. By stacking two MHA modules, the architecture is able to model increasingly complex dependencies across structured input components – namely, group characteristics, item features, contextual conditions, and evaluation criteria. Each layer incrementally enhances the semantic richness of the sequence representation by enabling higher-order feature interactions.

Unlike attention mechanisms in NLP, which operate over word sequences, our attention layers process structured embeddings corresponding to heterogeneous categorical fields. This design facilitates fine-grained reasoning suited for group-level recommendation tasks, where interactions span multiple semantic dimensions.

3.2.5 Cross-attention module

To extract a unified global representation for prediction, the architecture applies a Cross-Attention mechanism, where the first token in the sequence – corresponding to the `[CLS]` embedding – attends to all tokens, including itself. This mechanism enables the `[CLS]` token to aggregate information from the entire sequence, which includes the group, item, context, and criteria embeddings.

Let the output of the second MHA module be $Z^{(2)} \in \mathbb{R}^{(n+1)\times d}$, where the first token $Z_0^{(2)}$ corresponds to the special `[CLS]` vector. This token serves as the query in the Cross-Attention module, while the full sequence – including the `[CLS]` token – is used as both the key and value.

We compute the query, key, and value projections as follows:

$$Q_i^{(3)} = Z_0^{(2)} W_Q^{(3,i)} \tag{14}$$

$$K_i^{(3)} = Z^{(2)} W_K^{(3,i)} \tag{15}$$

$$V_i^{(3)} = Z^{(2)} W_V^{(3,i)} \tag{16}$$

where $Z_0^{(2)} \in \mathbb{R}^{1\times d}$ representing the `[CLS]` token used as the query. Each head $i \in \{1, \ldots, h\}$ projects inputs into d_h dimensional subspaces using learned matrices $W_Q^{(3,i)}$, $W_K^{(3,i)}$, $W_V^{(3,i)} \in \mathbb{R}^{d\times d_h}$. This yields $Q_i^{(3)} \in \mathbb{R}^{1\times d_h}$ and $K_i^{(3)}$, $V_i^{(3)} \in \mathbb{R}^{(n+1)\times d_h}$.

The scaled dot-product attention for each head is then computed as:

$$\text{head}_i^{(3)} = \text{Softmax}\left(\frac{Q_i^{(3)}(K_i^{(3)})^\top}{\sqrt{d_h}}\right) V_i^{(3)} \tag{17}$$

where $\text{head}_i^{(3)} \in \mathbb{R}^{1\times d_h}$ is the output of head i, computed by applying attention weights (via Softmax) to the value matrix. The scaling by $\sqrt{d_h}$ stabilizes training by preventing large dot-product magnitudes.

All head outputs are concatenated and projected back to the model dimension:

$$\mathcal{A}(Z^{(2)}) = \left[\text{head}_1^{(3)}, \ldots, \text{head}_h^{(3)}\right] W_O^{(3)} \tag{18}$$

where $\mathcal{A}(Z^{(2)}) \in \mathbb{R}^{1\times d}$ is the final cross-attention output. The h head outputs are concatenated and projected using $W_O^{(3)} \in \mathbb{R}^{hd_h\times d}$ to restore the model dimension.

Finally, a residual connection and *Layer Normalization* are applied to the `[CLS]` token to stabilize and refine the aggregated representation:

$$Z_{\text{CLS}} = \mathcal{LN}(Z_0^{(2)} + \mathcal{A}(Z^{(2)})) \tag{19}$$

The resulting vector $Z_{\text{CLS}} \in \mathbb{R}^{1\times d}$ encodes high-level, context-aware information aggregated from all structured input fields.

3.2.6 Cross network module

To explicitly model higher-order feature interactions, we apply a Cross Network [52] to Z_{CLS}. Letting $\mathbf{x}_0 = Z_{\text{CLS}}$, the Cross Network iteratively refines this vector through a series of transformations:

$$\mathbf{x}_{k+1} = \mathbf{x}_0 \cdot (\mathbf{w}_k^\top \mathbf{x}_k) + \mathbf{b}_k + \mathbf{x}_k, \quad \text{for } k = 0, \ldots, K-1 \tag{20}$$

where $\mathbf{x}_k \in \mathbb{R}^d$ is the input to the k-th cross layer, $\mathbf{w}_k \in \mathbb{R}^d$ and $\mathbf{b}_k \in \mathbb{R}$ are learnable parameters, and $\cdot$ denotes element-wise multiplication between the scalar projection $\mathbf{w}_k^\top \mathbf{x}_k$ and the base input $\mathbf{x}_0$.

This architecture enables the network to capture higher-order feature interactions in a parameter-efficient and interpretable manner by recursively applying cross terms over the original input. After K cross layers, the final output of the Cross Network is denoted as $Z_{\text{cross}} = \mathbf{x}_K$, where $Z_{\text{cross}} \in \mathbb{R}^d$. This cross-enhanced representation is then concatenated with the output of the linear embedding layer and fed into the fully connected neural network for the final prediction.

3.2.7 Fully connected layer and prediction layer

The outputs from the Linear Embedding Layer and the Cross Network Module are denoted as $Z_{\text{linear}} \in \mathbb{R}^d$ and $Z_{\text{cross}} \in \mathbb{R}^d$, respectively. These are concatenated to form the input to the deep neural network:

$$h^{(0)} = [Z_{\text{cross}}; Z_{\text{linear}}] \in \mathbb{R}^{2d} \tag{21}$$

This combined vector is then passed through a multi-layer perceptron composed of L layers. For each layer $\ell \in \{1, \ldots, L\}$, the transformation is defined as:

$$h^{(\ell)} = \sigma\left(W^{(\ell)} h^{(\ell-1)} + b^{(\ell)}\right) \tag{22}$$

where $h^{(\ell)} \in \mathbb{R}^{d_\ell}$ is the hidden representation at layer ℓ, $W^{(\ell)} \in \mathbb{R}^{d_{\ell-1} \times d_\ell}$ is the weight matrix, $b^{(\ell)} \in \mathbb{R}^{d_\ell}$ is the bias vector, and $\sigma(\cdot)$ denotes a non-linear activation function such as ReLU.

The final hidden layer output $h^{(L)} \in \mathbb{R}^{d_L}$ is projected to a scalar logit via:

$$\hat{R} = W_{\text{out}}^\top h^{(L)} + b_{\text{out}} \tag{23}$$

where $W_{\text{out}} \in \mathbb{R}^{d_L}$ and $b_{\text{out}} \in \mathbb{R}$ are the learnable parameters of the output layer, and $\hat{R} \in \mathbb{R}$ denotes the final predicted score or rating. The model is trained to minimize the difference between $\hat{R}$ and the true target value using a regression loss function, such as mean squared error.

This formulation ensures that the prediction $\hat{R}$ captures both the first-order effects (modeled by the linear embedding) and higher-order feature interactions (captured through the cross network and deep neural network), which are jointly optimized in an end-to-end learning framework.

3.3 Loss function and optimization

In our training process, we use the Mean Squared Error (*MSE*) as the loss function, which is mathematically defined as:

$$L(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{R}_i - R_i\right)^2 \tag{24}$$

where N is the total number of historical interaction data, $\hat{R}_i$ is the predicted value for the i-th sample, R_i is the true value for the i-th sample.

For the optimization algorithm, we used is *Adagrad*, which adapts the learning rate for each parameter individually based on the historical gradient information [9]. The update rule for *Adagrad* is given by:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{S_t + \epsilon}} \cdot g_t \tag{25}$$

where θ_t is the parameter at time step t, η is the global learning rate, S_t is the sum of squares of past gradients up to time t, ϵ is a small constant added to avoid division by zero, s_t is the gradient of the loss function at time t.

Overall, our architecture allows CA-MCGRS to dynamically adapt to different contexts and criteria, ensuring that recommendations are both highly relevant and precise by leveraging the MHA mechanism. In the next section, we present experiments designed to assess its performance.

Table 2 Statistics of ITM-Rec and MovieLens 100K datasets

Dataset	Object	Quantity	Nb of Item	Contexts	Criteria	Rating Scale	Data Sparsity	Nb of Rating
ITM Rec	Individual	454	70	Class, Semester, Lockdown	App, Data, Ease	[1,5]	83.54%	5230
	Group	143					88.84%	1117
Movielen 100k	Individual	943	1682	TimeOfDay, DayType, Season	Storyline, Visuals, Emotion	[1,5]	93.70%	100.000
	Group	250					83.55%	

4 Experimental setup

In this section, we introduce the dataset utilized to assess the performance of our approach and describe the baseline models used for comparison. Finally, we provide a detailed analysis of the experimental results.

4.1 Dataset

To evaluate the effectiveness of our CA-MCGRS model in group recommendation scenarios, we use group-level data from two datasets that capture ratings under diverse contexts and criteria. A summary of the group-level data statistics is provided in Table 2.

ITM-Rec Dataset [60]: This dataset contains ratings provided by 143 student groups from the Illinois Institute of Technology. Each group rated 70 educational project topics based on three evaluation criteria: Applicability, Data Quality, and Ease of Use. Ratings are contextualized using academic variables such as Class, Semester, and Lockdown Status. The group data exhibits a sparsity of 88.84% and includes 1,117 ratings.

Group-Enhanced MovieLens 100K: The original MovieLens 100K dataset[1] does not include group-level ratings, which are essential for evaluating group recommender systems. To address this limitation, we created a group-enhanced version by simulating group interactions. Users were clustered into 250 synthetic groups using KMeans based on demographic attributes (age, occupation) and genre preferences. Each group provides collective feedback on movies, aggregated from individual preferences. While no explicit contextual or multi-criteria information is attached to the group ratings, the dataset includes 100,000 ratings with a sparsity of 83.55%. We further enriched the individual-level data with contextual features TimeOfDay, DayType, Season) derived from timestamps and simulated multi-criteria ratings (Storyline, Visuals, Emotion) by adding Gaussian noise to the original scores [53]. This enhanced dataset enables context-aware and multi-criteria recommender system experiments.

4.2 Baseline models

To evaluate our model's effectiveness, we compare its performance with several established baseline methods:

- *AutoInt* [48]: Automatic Feature Interaction Learning via Self-Attentive Neural Networks captures high-order feature interactions using multi-head self-attention. By projecting

[1] https://grouplens.org/datasets/movielens/100k/

features into a shared space and applying attention layers with residual connections, *AutoInt* efficiently models complex dependencies for click-through rate (CTR) prediction.

- *DCN* [52]: Deep Cross Network (*DCN*) captures feature interactions by utilizing a cross network structure, which explicitly models the crossover relationships between different feature dimensions at multiple representation levels. The cross network applies element-wise feature interactions recursively across layers, allowing it to efficiently capture both low-order and high-order interactions without requiring deep architectures.
- *CCPM* [27]: Convolutional Click Prediction Model introduces a novel click prediction approach leveraging convolutional neural networks. The proposed model effectively captures both local and global dependencies among features by applying convolution operations on embedding layers, significantly improving the accuracy of click-through rate predictions.
- *DIFM* [28]: A Dual Input-aware Factorization Machine (*DIFM*) for CTR Prediction proposes a model that dynamically adjusts feature embeddings for each input instance. Unlike traditional Factorization Machines (FM), *DIFM* employs bit-wise and vector-wise reweighting with self-attention and deep neural networks to capture complex feature interactions.
- *PNN* [40]: The Product-based Neural Network (*PNN*) model, is designed to predict user responses in online advertising. It incorporates an embedding layer to learn distributed representations of categorical data, effectively capturing both feature interactions and non-linearities.
- *IFM* [57]: The Input-aware Factorization Machine (*IFM*) improves traditional FMs by dynamically adjusting feature representations based on individual input instances. Unlike standard FMs, which use fixed feature embeddings, *IFM* introduces input-aware factors learned through a neural network, reducing noise and enhancing predictive accuracy.
- *AFM* [54]: The Attentional Factorization Machine (*AFM*) enhances traditional FMs by dynamically learning the importance of different feature interactions using a neural attention mechanism. *AFM* assigns varying weights to interactions based on their relevance, reducing noise and improving predictive performance.
- *LS-PLM* [10]: The Large Scale Piece-wise Linear Model (*LS-PLM*) is a scalable CTR prediction solution for sparse data. It enhances feature selection with different regularizers and optimizes efficiently using a quasi-Newton method. Designed for industrial deployment, it runs on distributed systems, minimizing manual feature engineering.
- *DeepFM* [11]: *DeepFM* is a hybrid model that seamlessly integrates FM with deep neural networks (DNN). The FM component efficiently captures low-order feature interactions, while the deep neural network leverages multiple nonlinear activation layers to model complex, high-order feature interactions. By learning both low-order and high-order patterns simultaneously.
- *FiBiNET* [14]: Fature Importance & Bilinear feature Interaction Network (*FiBiNET*) enhances feature representation by using both a Squeeze-Excitation (SE) network and bilinear interaction layers. The SE network adaptively recalibrates feature importance by learning feature-wise dependencies, while the bilinear interaction layer captures intricate pairwise feature interactions in a more flexible manner.
- *NFM* [13]: Neural Factorization Machines (*NFM*) extend traditional FM by incorporating a neural network layer to model higher-order feature interactions. The FM component efficiently captures second-order feature interactions, while the deep neural network extracts more complex dependencies among features.

- *ONN* [56]: Operation-aware Neural Networks (*ONN*) improve feature interaction modeling by explicitly incorporating predefined mathematical operations, such as addition, multiplication, and subtraction. Instead of relying solely on learned transformations, *ONN* utilizes operation-aware representations to enhance user response prediction.
- *WDL* [5]: Wide & Deep Learning (*WDL*) is a hybrid model designed to capture both memorization and generalization in predictive tasks. The wide linear model learns explicit feature interactions by using handcrafted cross-product features, enabling fast and interpretable predictions. The deep neural network component captures complex patterns and high-order feature interactions through nonlinear transformations.
- *xDeepFM* [24]: The eXtreme Deep Factorization Machine (*xDeepFM*) integrates a Compressed Interaction Network (CIN) to capture explicit vector-wise feature interactions. Unlike traditional DNN-based models, CIN explicitly models feature interactions at different levels without relying on manually crafted features.

The experiments were conducted using the PyTorch framework [39] and the DeepCTR-Torch library [47], both of which provide efficient tools for deep learning model development and experimentation. For each model, we relied on the default hyperparameters, as they were carefully pre-optimized to ensure stable performance and reliable results. This strategy allowed us to focus on evaluating the effectiveness of different model architectures without the need for extensive hyperparameter tuning.

Table 3 summarizes the key hyperparameters used in our MHA-based CA-MCGRS architecture. The model employs 4 attention heads to capture diverse interactions among input features, with all categorical and contextual variables embedded into 8-dimensional vectors. These embeddings are processed through a DNN composed of six hidden layers with sizes (256, 128, 64, 32, 16, 8), enabling progressive feature abstraction. A dropout rate of 0.42 is applied in both the attention and DNN layers to prevent overfitting, and L2 regularization with a weight decay of 1×10^{-5} is applied to linear, embedding, and DNN components. The ReLU activation function is used in all hidden layers for its efficiency and stability in deep neural architectures. The model is trained using the Adam optimizer, selected for its adaptive learning capabilities and robust convergence, with a learning rate of 0.001 for up to 50 epochs and a batch size of 256. The Mean Squared Error (MSE) is used as the loss function, as it aligns with our regression-based objective for rating prediction. All hyperparameters were

Table 3 Hyperparameters of the MHA Model

Hyperparameter	Default Value
Number of attention heads in MHA	4
Hidden layer sizes	(256, 128, 64, 32, 16, 8)
L2 regularization for linear layer	1×10^{-5}
L2 regularization for embedding layer	1×10^{-5}
Learning rate for optimizer	0.001
Optimization algorithm	`Adam`
Loss function used in training	`mse` (Mean Squared Error)
Maximum number of training epochs	50
Dropout rate used in attention and DNN layers	0.42
Embedding dimension for all features	8
Batch size used during training	256

tuned via grid search using a held-out validation set, with selection based on minimizing validation RMSE. Based on the feature space and group structure, the number of trainable parameters varies across datasets, with 69,766 parameters for MovieLens-100k and 51,318 parameters for ITM-Rec. These differences reflect variations in the number of unique features and embedding dimensions. Non-trainable elements such as dropout layers and internal normalization statistics support model stability but are not updated during training.

4.3 Evaluation metrics

We evaluate the performance of our deep neural architecture using four key metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Precision at rank k (Precision@k), and Normalized Discounted Cumulative Gain at rank k (NDCG@k). MAE and RMSE are standard regression metrics that quantify the discrepancy between predicted and actual values [46]. MAE calculates the average absolute difference, while RMSE penalizes larger errors more heavily, making it more sensitive to outliers. Both metrics are used to assess prediction accuracy, where lower values indicate better model performance.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|R_i - \hat{R}_i\right| \tag{26}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(R_i - \hat{R}_i\right)^2} \tag{27}$$

where n is the number of observations, R_i is the actual value, and $\hat{R}_i$ is the predicted value.

In addition to error-based metrics, we incorporate ranking-based metrics – Precision@k and NDCG@k – to evaluate the relevance and ordering of top-k recommendations. Precision@k measures the proportion of relevant items among the top-k results:

$$\text{Precision@}k = \frac{1}{k}\sum_{i=1}^{k}\text{rel}_i \tag{28}$$

where $\text{rel}_i = 1$ if the item at rank i is relevant, and 0 otherwise.

NDCG@k evaluates the quality of a ranked list by assigning higher importance to relevant items that appear near the top. It is computed by normalizing the Discounted Cumulative Gain (DCG@k) using the Ideal DCG (IDCG@k):

$$\text{NDCG@}k = \frac{1}{\text{IDCG@}k}\sum_{i=1}^{k}\frac{2^{\text{rel}_i}-1}{\log_2(i+1)} \tag{29}$$

where rel_i denotes the relevance of the item at rank i. The ideal DCG, denoted as IDCG@k, is calculated by sorting the relevance scores in descending order and applying the same DCG formula:

$$\text{IDCG@}k = \sum_{i=1}^{k}\frac{2^{\text{rel}_i^*}-1}{\log_2(i+1)} \tag{30}$$

where rel_i^* refers to the relevance scores in the ideal (best possible) ranking. By dividing DCG@k by IDCG@k, the NDCG@k score is normalized between 0 and 1, with 1 indicating a perfect ranking in which all relevant items are ranked in the optimal order.

Together, these metrics provide a comprehensive view of model performance, evaluating both prediction accuracy and recommendation quality. In the following sections, we present experimental results using these metrics to compare our model against baseline approaches. In the following sections, we present the results of our experiments, comparing our model against baseline approaches using these metrics to highlight its advantages.

5 Experimental results and analysis

We now present the experimental results validating the effectiveness of our MHA-based architecture for the CA-MCGRS. The subsequent analysis, conducted on the ITM-Rec and MovieLens-100k datasets, compares MHA against established baselines. The experiments aim to firstly illustrate the performance gains achieved by systematically adding multi-criteria and contextual dimensions to group recommendations, and secondly, to explore the nuanced impact of various single contextual elements and criteria configurations on recommendation quality.

5.1 Experiment 1: performance evolution from GRS to context-aware MCGRS

The first set of experiments aimed to investigate the performance evolution of recommendation models – particularly our MHA architecture – as the complexity of the recommendation task increased. We systematically evaluated model performance by starting from a basic GRS setting, which does not incorporate contextual or multi-criteria information. We then extended the task to a multi-criteria setting, and finally to a multi-criteria, multi-context setting (MCGRS-MC).

The results on the ITM-Rec dataset are presented in Table 4. In the GRS setting, our MHA model (RMSE: 1.1622, MAE: 1.0195) did not achieve the lowest error rates. *AutoInt* performed best in this setting, obtaining an RMSE of 0.9436 and a MAE of 0.7304. This suggests that, in the absence of rich multi-criteria and contextual features, which MHA is specifically designed to leverage, its architectural complexity may not provide a clear advantage over models optimized for simpler recommendation tasks. Regarding ranking performance, *LS-PLM* achieved the highest NDCG@5, while NFM led in terms of Precision@5.

Upon transitioning to the MCGRS setting, which includes multi-criteria information without explicit context, MHA demonstrated a substantial improvement, achieving the best RMSE (0.7843) and MAE (0.6144) among all models. This highlights the effectiveness of the MHA mechanism in capturing and weighting multiple criteria. *LS-PLM* led in NDCG@5, and *NFM* remained strong in Precision@5. In the most complex MCGRS-MC setting – incorporating both multi-criteria ratings and multiple contextual features – MHA maintained its lead, achieving the best RMSE (0.8029) and MAE (0.6182). Although the error slightly increased compared to the MCGRS setting, MHA still outperformed strong baselines like *AutoInt* (RMSE: 0.8180, MAE: 0.6330). This suggests that while MHA effectively integrates contextual information, the combined complexity may dampen the additional gains. *PNN* demonstrated the best Precision@5 in this setting.

Table 5 presents the results on the MovieLens-100k dataset, which largely reflect the trends observed in the ITM-Rec results. In the GRS setting, the MHA model produced competitive results (RMSE: 0.9440, MAE: 0.7471), although *xDeepFM* slightly outperformed it with an RMSE of 0.9411 and a MAE of 0.7392. In terms of ranking metrics, *IFM* achieved the highest NDCG@5, while *LS-PLM* led in Precision@5, further highlighting MHA's relatively lower

Table 4 Performance comparison of recommendation models on ITM-Rec dataset across four experimental settings: GRS without contextual or criteria-based information, MCGRS without context, MCGRS-MC with multiple contextual factors

Model	GRS				MCGRS				MCGRS-MC			
	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5
AFM	1.1191	0.9283	0.9541	0.4130	0.9874	0.8223	0.9611	0.4226	1.0007	0.8302	0.9598	0.4205
AutoInt	**0.9436**	**0.7304**	0.9635	0.4192	0.8098	0.6210	0.9734	0.4281	0.8180	0.6330	0.9723	0.4288
CCPM	1.2312	1.0155	0.9330	0.4000	1.0609	0.8509	0.9552	0.4192	1.1022	0.8724	0.9532	0.4164
DCN	0.9473	0.7401	0.9639	0.4199	0.8129	0.6271	0.9722	0.4295	0.8228	0.6395	0.9724	0.4288
DeepFM	0.9450	0.7363	0.9633	0.4199	0.8177	0.6307	0.9727	0.4295	0.8147	0.6287	0.9731	0.4288
DIFM	1.2818	1.0415	0.9262	0.3973	1.2370	1.0076	0.9329	0.4007	1.2354	1.0097	0.9337	0.4014
FiBiNET	0.9625	0.7605	0.9610	0.4219	0.8471	0.6688	0.9700	0.4267	0.8353	0.6572	0.9691	0.4295
NFM	0.9596	0.7501	0.9620	**0.4226**	0.8294	0.6389	0.9699	**0.4301**	0.8213	0.6350	0.9698	0.4295
ONN	1.0400	0.8359	0.9541	0.4171	0.8924	0.7023	0.9626	0.4253	0.8873	0.6908	0.9644	0.4267
PNN	0.9485	0.7332	0.9613	0.4212	0.8233	0.6272	0.9679	0.4288	0.8271	0.6515	0.9692	**0.4315**
WDL	0.9449	0.7363	0.9638	0.4205	0.8169	0.6308	0.9720	0.4295	0.8134	0.6277	0.9728	0.4288
IFM	1.1966	1.0367	0.9627	0.4171	1.1559	0.9294	0.9545	0.4103	1.1744	0.9530	0.9463	0.4075
xDeepFM	0.9499	0.7436	0.9647	0.4192	0.8113	0.6239	0.9725	0.4288	0.8164	0.6305	0.9723	0.4288
LS-PLM	1.5065	1.2690	**0.9652**	0.4151	0.8725	0.6924	**0.9750**	0.4260	0.8753	0.7060	**0.9743**	0.4281
MHA	1.1622	1.0195	0.9601	0.4171	**0.7843**	**0.6144**	0.9744	0.4281	**0.8029**	**0.6182**	0.9727	0.4281

Each setting reports RMSE, MAE, N@5(NDCG@5), and P@5(Precision@5). Best results are underlined and bolded

Table 5 Performance of recommendation models on MovieLens under four settings: GRS (no context/criteria), MCGRS (multi-criteria only), MCGRS-MC (multi-criteria + multi-context)

Model	GRS				MCGRS				MCGRS-MC			
	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5
AFM	0.9485	0.7500	0.9097	0.8128	0.8948	0.7105	0.9307	0.8520	0.8915	**0.7039**	**0.9393**	0.8600
AutoInt	0.9467	0.7519	0.9167	0.8120	0.8963	0.7062	0.9339	0.8512	0.8961	0.7128	0.9310	0.8552
CCPM	0.9495	0.7432	0.9106	0.8096	0.9231	0.7237	0.9227	0.8360	0.9148	0.7231	0.9270	0.8448
DCN	0.9466	0.7475	0.9136	0.8112	0.8925	0.7054	0.9283	0.8448	0.8922	0.7084	0.9350	0.8560
DeepFM	0.9459	0.7474	0.9145	0.8104	0.8927	0.7086	0.9333	0.8544	0.8965	0.7048	0.9307	0.8560
DIFM	0.9617	0.7526	0.9081	0.8072	0.9027	0.7118	0.9348	0.8576	0.9110	0.7124	0.9370	0.8544
FiBiNET	0.9451	0.7448	0.9147	0.8240	0.9002	0.7106	0.9321	**0.8584**	0.8970	0.7062	0.9319	0.8592
NFM	0.9534	0.7494	0.9142	0.8064	0.9024	0.7122	0.9333	0.8544	0.8964	0.7048	0.9335	0.8544
ONN	1.0442	0.8241	0.8868	0.7496	0.9865	0.7771	0.9084	0.8016	0.9968	0.7850	0.9045	0.8008
PNN	0.9587	0.7488	0.9116	0.8112	0.9011	0.7101	0.9281	0.8512	0.9005	0.7083	0.9298	**0.8624**
WDL	0.9462	0.7479	0.9156	0.8120	0.8933	0.7101	0.9336	0.8576	0.8960	0.7044	0.9322	0.8568
xDeepFM	**0.9411**	**0.7392**	0.9149	0.8216	0.8905	0.7054	0.9309	0.8544	0.8906	0.7074	0.9332	0.8616
IFM	0.9429	0.7456	0.9143	**0.8352**	0.8942	0.7062	**0.9357**	0.8560	0.8906	0.7077	0.9303	0.8528
LS-PLM	0.9458	0.7487	**0.9165**	0.8224	0.8936	0.7060	0.9321	0.8568	0.8948	0.7088	0.9362	0.8552
MHA	0.9440	0.7471	0.9143	0.8176	**0.8895**	**0.7038**	0.9344	**0.8584**	**0.8899**	0.7058	0.9347	0.8576

Best scores per metric are underlined

effectiveness in simpler recommendation scenarios. When multi-criteria information was introduced in the MCGRS setting, MHA demonstrated its strength by achieving the lowest RMSE (0.8895) and MAE (0.7038) among all models. It also shared the best Precision@5 score (0.8584) with *FiBiNET*, confirming its capability to effectively leverage multi-criteria data for both accuracy and ranking relevance. In the more complex MCGRS-MC setting, which incorporates both multi-criteria and contextual features, MHA continued to perform strongly, achieving the best RMSE (0.8899) and a highly competitive MAE (0.7058), closely following *AFM* (0.7039). *PNN* recorded the top Precision@5 in this setting. These findings highlight MHA's robustness in maintaining high prediction accuracy even when handling the increased complexity introduced by multiple types of input information.

Across both datasets, a consistent trend emerged: the MHA model exhibited its most significant performance improvements when transitioning from the GRS to the MCGRS setting. This highlights the effectiveness of the MHA mechanism in capturing the interactions and relative importance of multiple criteria within group recommendation tasks. While the addition of contextual features in the MCGRS-MC setting generally maintained or modestly enhanced predictive accuracy, the performance gains were less pronounced than those achieved by introducing multi-criteria information. This suggests that contextual data provides incremental benefits, especially once multi-criteria ratings are already in use. Additionally, although baseline models like *AutoInt* and *xDeepFM* performed well in the simpler GRS scenarios, MHA consistently outperformed them as the task complexity increased. These results underscore the scalability and adaptability of the MHA architecture in handling rich, structured input data. Building upon these findings, we further investigated how the integration of individual contextual factors – alongside multi-criteria ratings – affects model performance, with a particular focus on the adaptability of the MHA architecture across varying context-aware scenarios.

5.2 Experiment 2: integration of context and multi-criteria information

The second phase of our experimental evaluation focused on assessing how different models, and our proposed MHA architecture in particular, perform when single contextual factors are integrated alongside multi-criteria ratings. This design enables a more granular understanding of context sensitivity. For the ITM-Rec dataset, we investigated three distinct single-context scenarios: Class (MCGRS-SC-C), Semester (MCGRS-SC-S), and Lockdown (MCGRS-SC-L). Similarly, for the MovieLens-100k dataset, we considered the contexts of TimeOfDay (MCGRS-SC-T), DayType (MCGRS-SC-D), and Season (MCGRS-SC-S). All experimental settings included multi-criteria rating information. In all settings, multi-criteria information is preserved to ensure comparability.

Detailed results for the ITM-Rec dataset under single-context integration are presented in Table 6. Across all three single-context configurations, the MHA model consistently outperforms all baselines in terms of RMSE and MAE across the three single-context scenarios. In the Class context (MCGRS-SC-C), MHA achieves an RMSE of 0.7621 and an MAE of 0.5866, significantly outperforming the next best models such as *PNN* (RMSE: 0.8133, MAE: 0.6218) and *DeepFM* (RMSE: 0.8146, MAE: 0.6273). In the Semester setting (MCGRS-SC-S), MHA once again leads with an RMSE of 0.7671 and MAE of 0.5888, surpassing models like *AutoInt* and *xDeepFM*. The most notable performance is observed in the Lockdown context (MCGRS-SC-L), where MHA achieves its lowest errors with an RMSE of 0.7493 and MAE of 0.5805 – clearly outperforming models such as *PNN* and *DeepFM*. This highlights

Table 6 Performance comparison of recommendation models across four experimental settings: MCGRS-SC-C with Single Contextual factors (Class context in this case); MCGRS-SC-S, a Multi-Criteria Group Recommender System that considers only Single-Context information related to the Semester context; MCGRS-SC-L, which follows the same approach but focuses on the Lockdown context

Model	MCGRS-SC-C				MCGRS-SC-S				MCGRS-SC-L			
	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5
AFM	1.0140	0.8636	0.9703	0.4233	0.9971	0.8259	0.9590	0.4212	0.9979	0.8239	0.9621	0.4212
AutoInt	0.8153	0.6299	0.9725	0.4295	0.8116	0.6248	0.9723	0.4288	0.8163	0.6308	0.9723	0.4295
CCPM	1.1103	0.8793	0.9496	0.4158	1.1385	0.8908	0.9475	0.4151	1.1169	0.8876	0.9427	0.4116
DCN	0.8245	0.6529	0.9713	0.4295	0.8149	0.6302	0.9732	0.4295	0.8244	0.6529	0.9716	0.4295
DeepFM	0.8146	0.6273	0.9730	0.4288	0.8162	0.6295	0.9723	0.4288	0.8146	0.6271	0.9730	0.4288
DIFM	1.2347	1.0105	0.9327	0.4027	1.2375	1.0132	0.9322	0.4014	1.2377	1.0137	0.9337	0.4014
FiBiNET	0.8415	0.6618	0.9699	0.4267	0.8400	0.6597	0.9699	0.4281	0.8388	0.6591	0.9699	0.4267
NFM	0.8258	0.6358	0.9691	**0.4308**	0.8245	0.6389	0.9713	**0.4301**	0.8292	0.6361	0.9703	**0.4308**
ONN	0.8924	0.6969	0.9652	0.4295	0.9045	0.7033	0.9675	0.4281	0.8916	0.6961	0.9634	0.4301
PNN	0.8133	0.6218	0.9690	0.4295	0.8256	0.6332	0.9662	0.4281	0.8131	0.6217	0.9690	0.4295
WDL	0.8130	0.6264	0.9727	0.4288	0.8147	0.6285	0.9722	0.4288	0.8129	0.6263	0.9727	0.4288
xDeepFM	0.8177	0.6321	0.9724	0.4288	0.8128	0.6252	0.9730	0.4288	0.8176	0.6321	0.9724	0.4288
IFM	1.1631	0.9438	0.9503	0.4110	1.1639	0.9405	0.9511	0.4103	1.1678	0.9453	0.9497	0.4123
LS-PLM	0.8705	0.6944	**0.9749**	0.4274	0.8708	0.6945	**0.9746**	0.4267	0.8707	0.6945	**0.9749**	0.4274
MHA	**0.7621**	**0.5866**	0.9721	0.4281	**0.7671**	**0.5888**	0.9718	0.4288	**0.7493**	**0.5805**	0.9737	0.4288

Table 7 Performance comparison for MCGRS-SC-T (TimeOfDay), MCGRS-SC-D (DayType), and MCGRS-SC-Se (Season) settings

Model	MCGRS-SC-T				MCGRS-SC-D				MCGRS-SC-Se			
	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5	RMSE	MAE	N@5	P@5
AFM	0.8907	0.7071	0.9334	**0.8608**	0.8944	0.7071	0.9346	0.8544	0.8899	0.7042	0.9335	0.8520
AutoInt	0.9038	0.7237	0.9341	0.8592	0.9034	0.7228	0.9328	0.8616	0.8924	0.7050	**0.9351**	0.8544
CCPM	0.9293	0.7355	0.9269	0.8400	0.9063	0.7117	**0.9359**	0.8472	0.8898	**0.7027**	0.9332	0.8536
DCN	0.8920	0.7034	**0.9361**	0.8568	0.8922	**0.7023**	0.9344	0.8536	0.9031	0.7047	0.9260	0.8544
DeepFM	0.8943	0.7043	0.9301	0.8584	0.8941	0.7127	0.9324	0.8568	0.8938	0.7047	0.9312	0.8568
DIFM	0.9001	0.7067	0.9311	0.8568	0.9169	0.7155	0.9297	0.8544	0.9014	0.7069	0.9343	0.8560
FiBiNET	0.8913	**0.7015**	0.9330	0.8576	0.8900	0.6997	0.9295	0.8632	0.8967	0.7080	0.9338	0.8528
NFM	0.8997	0.7178	0.9319	0.8536	0.8939	0.7031	0.9339	**0.8648**	0.8999	0.7179	0.9328	0.8576
ONN	0.9843	0.7779	0.9150	0.8216	0.9753	0.7719	0.9093	0.8224	0.9766	0.7713	0.9198	0.8296
PNN	0.9027	0.7071	0.9283	0.8544	0.9051	0.7121	0.9298	0.8520	0.8967	0.7037	0.9311	0.8448
WDL	0.8945	0.7042	0.9316	0.8600	0.8937	0.7126	0.9348	0.8560	0.8939	0.7044	0.9312	**0.8592**
xDeepFM	0.8911	0.7080	0.9331	0.8592	0.8906	0.7070	0.9352	0.8608	**0.8897**	0.7028	0.9326	0.8552
IFM	0.8939	0.7027	0.9349	0.8560	0.8918	0.7029	0.9330	0.8528	0.8940	0.7031	0.9322	0.8512
LS-PLM	0.8959	0.7090	**0.9361**	0.8552	0.8960	0.7101	0.9351	0.8536	0.8935	0.7063	0.9310	0.8504
MHA	**0.8895**	0.7054	0.9351	0.8600	**0.8896**	0.7060	0.9357	0.8584	0.8898	0.7067	0.9345	**0.8592**

the Lockdown variable as a highly informative contextual factor and suggests that MHA's attention mechanism effectively captures such impactful signals.

While MHA leads in predictive accuracy, the ranking metrics tell a more nuanced story. In terms of NDCG@5, *LS-PLM* consistently achieves the highest scores across all three contexts, while *NFM* secures top Precision@5 values. MHA remains competitive but does not lead these ranking-based metrics. For example, in the Lockdown setting, MHA attains a Precision@5 of 0.4288, close to *NFM*'s 0.4308 but not the best. This divergence suggests that although MHA predicts ratings with high precision, other models may be better suited for top-k recommendation tasks in certain settings.

The results from the MovieLens-100k dataset, shown in Table 7, further confirm MHA's strength in predictive accuracy. Under the TimeOfDay context (MCGRS-SC-T), MHA yields the best RMSE (0.8895) and a highly competitive MAE (0.7054), narrowly trailing *FiBiNET* (0.7015). For ranking metrics in this context, *DCN* and *LS-PLM* tie for the best NDCG@5 (0.9361), while *AFM* leads in Precision@5 (0.8608), slightly ahead of MHA (0.8600). In the DayType scenario (MCGRS-SC-D), MHA again records the lowest RMSE (0.8896), with a close MAE (0.7060) compared to DCN (0.7023). CCPM and NFM perform best in NDCG@5 and Precision@5 respectively. For the Season context (MCGRS-SC-Se), MHA's RMSE (0.8898) is nearly equal to *xDeepFM*'s best score (0.8897), while its MAE (0.7067) remains competitive. In this case, MHA and *WDL* both lead in Precision@5 with a score of 0.8592. These findings highlight MHA's adaptability in processing various contextual dimensions, although other models occasionally outperform it on fine-grained ranking tasks.

Synthesizing the findings across both datasets, several key observations emerge regarding the integration of single contextual factors with multi-criteria information. First, MHA demonstrates clear superiority in predictive accuracy across nearly all single-context scenarios. This advantage is particularly notable in the ITM-Rec dataset's Lockdown scenario, where MHA achieves an RMSE of 0.7493 and MAE of 0.5805 – well ahead of all competing models. Second, the results confirm that the informativeness of context varies. The Lockdown variable, for example, appears especially impactful, suggesting that MHA can identify and leverage critical contextual signals when they are present. Third, while MHA maintains top-tier performance in RMSE and MAE, it does not consistently lead in ranking metrics. Baseline models like *LS-PLM*, *NFM*, and *AFM* often dominate NDCG@5 and Precision@5 depending on the specific context. This indicates a possible trade-off between prediction precision and rank ordering performance. Finally, MHA's consistently strong performance across both educational and temporal contexts confirms its robustness and versatility. However, future improvements could involve adapting the architecture to explicitly optimize for ranking objectives, possibly through hybrid loss functions or attention modules tailored for top-k relevance.

5.3 Computational efficiency and training dynamics

Beyond predictive accuracy and ranking quality, the computational efficiency and training behavior of recommendation models are critical for their practical deployment. All models in this study were trained and evaluated under a controlled computational environment to ensure fair comparisons. This consistent setup eliminates external sources of variability and allows us to attribute observed differences strictly to the models' architectural characteristics.

To evaluate computational efficiency, we measured the average training time per model on both the ITM-Rec and MovieLens-100k datasets. The results, shown in Figure 2, reveal substantial variability in training time across models and datasets.

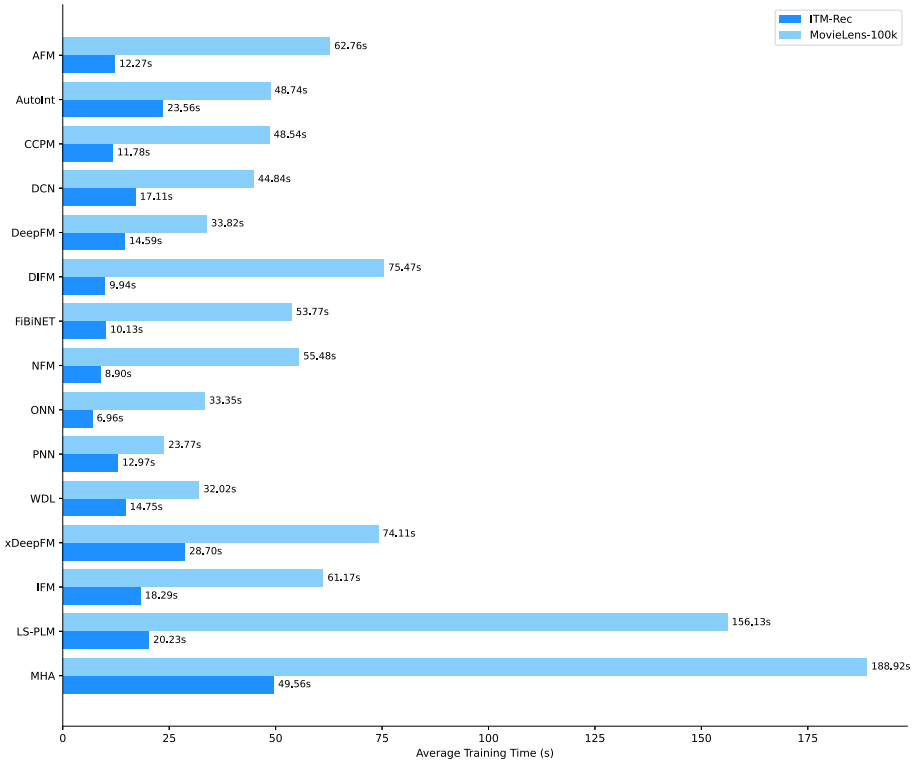


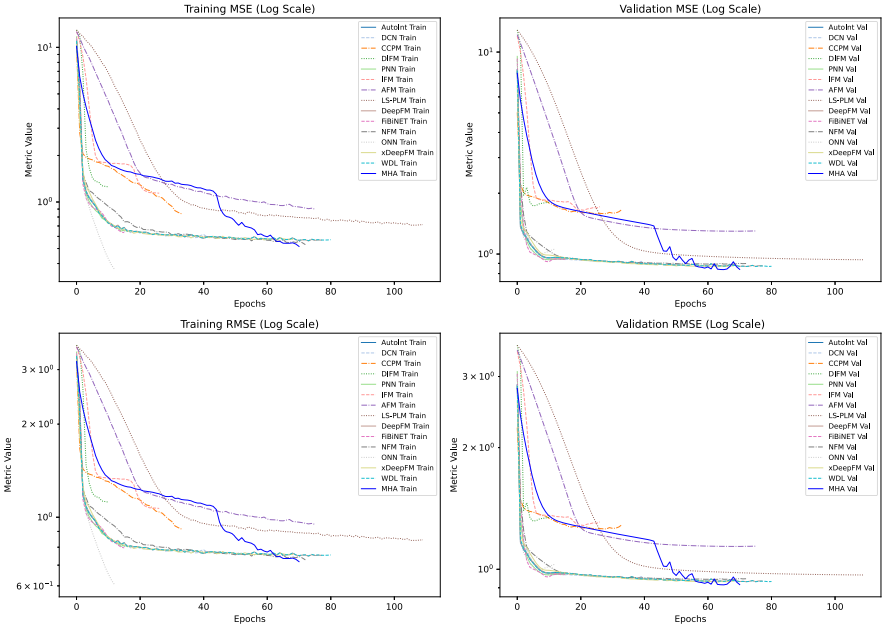
Fig. 2 Comparison of average training time across different methods

On the MovieLens-100k dataset, the most computationally efficient models include *ONN* (33.35s), *PNN* (23.77s), and *WDL* (32.02s), followed by *DeepFM* (33.82s). More complex models like *AutoInt* (48.74s), *CCPM* (48.54s), and *FiBiNET* (53.77s) require longer training durations. The highest training times are observed for *DIFM* (75.47s), *xDeepFM* (74.11s), *AFM* (62.76s), and MHA (188.92s), reflecting their deeper architectures and higher-order interactions. *LS-PLM* (156.13s) and *IFM* (61.17s) also exhibit significant computational cost.

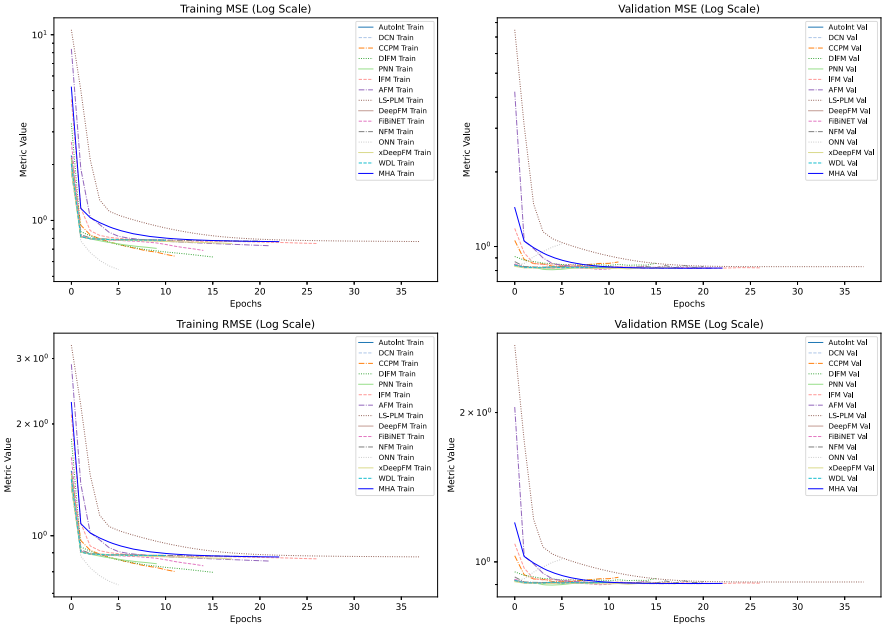
On the ITM-Rec dataset, training times are substantially lower. The fastest models are *ONN* (6.96s), *NFM* (8.90s), *DIFM* (9.94s), *FiBiNET* (10.13s), and *CCPM* (11.78s). Mid-range times include *AFM* (12.27s), *PNN* (12.97s), and *WDL* (14.75s). Models such as *DCN* (17.11s), *IFM* (18.29s), and *LS-PLM* (20.23s) show slightly higher costs. The most time-consuming models on this dataset are *AutoInt* (23.56s), *xDeepFM* (28.70s), and our proposed MHA model (49.56s), which reflects its computational complexity due to MHA mechanisms.

Overall, models such as *ONN*, *DIFM*, and *FiBiNET* exhibit consistent efficiency across both datasets. Although MHA has the highest training time, it provides significantly better predictive accuracy and generalization as previously demonstrated. These results emphasize the practical trade-off between computational efficiency and recommendation quality in real-world deployments.

To complement the training time analysis, Figure 3 presents the evolution of training and validation metrics – MSE and RMSE – over epochs for both the ITM-Rec and MovieLens-100k datasets. Across both datasets, the MHA model consistently demonstrates superior learning dynamics. It shows smooth, monotonic convergence in training loss and maintains



(a) ITM-Rec Dataset



(b) MovieLens-100k Dataset

Fig. 3 Training and validation metrics MSE and RMSE over epochs. Each plot includes Training MSE, Validation MSE, Training RMSE, and Validation RMSE

consistently lower validation errors compared to most baselines. In particular, its validation RMSE and MSE curves remain stable and low throughout training, indicating strong generalization capacity and resistance to overfitting. This is especially notable in the ITM-Rec dataset, where MHA's validation loss flattens early and stays significantly below that of competing models.

In contrast, models such as *DeepFM*, *FiBiNET*, and *AutoInt* exhibit steeper training loss reductions but their validation curves tend to plateau early or show signs of divergence. This behavior suggests these models may overfit to the training data, struggling to generalize in more complex or sparse recommendation contexts. For instance, *FiBiNET* and *AutoInt* show promising training RMSEs, but their validation RMSE curves either plateau at a higher level or fluctuate, indicating reduced robustness. Additionally, models like *xDeepFM*, *LS-PLM*, and *AFM* display slower convergence and higher overall validation error, possibly due to architectural limitations or sensitivity to hyperparameter configurations in multi-criteria and context-enriched scenarios.

One particularly compelling pattern is the small gap between MHA's training and validation curves, visible across both datasets and for both MSE and RMSE. This suggests that MHA not only fits the training data well but also avoids overfitting, maintaining high performance on unseen test data. The model's attention mechanism appears to effectively focus on relevant contextual and multi-criteria interactions, leading to both better learning stability and generalization. Taken together, these learning dynamics underscore the strength of the CA-MCGRS architecture. Despite requiring longer training times, MHA achieves more stable convergence and lower generalization error, making it well-suited for real-world recommendation scenarios where model robustness and reliability are critical.

6 Discussion

Despite the promising findings presented, this study is not without its limitations, which in turn illuminate several promising avenues for future research. One primary consideration is the computational scalability of the proposed MHA architecture. As evidenced by our analysis of training times (Figure 2), the MHA mechanisms, while crucial for achieving high predictive accuracy, do incur greater computational demands compared to simpler models. This can present challenges when deploying such systems in real-time environments dealing with extremely large user bases, numerous group interactions, or extensive item catalogs. Future investigations should therefore prioritize the exploration of model optimization techniques. Strategies such as knowledge distillation to smaller student models, parameter quantization, or the development of more computationally efficient attention variants could significantly enhance the system's throughput and reduce latency, making it more viable for large-scale, time-sensitive applications.

Another important observation from our experiments, particularly evident in the single-context analyses of Experiment 2 (Tables 6 and 7), is the nuanced relationship between predictive accuracy and top-k ranking performance. While our MHA model consistently excelled in minimizing overall prediction errors (RMSE and MAE), its dominance was less pronounced in ranking-specific metrics like NDCG@5 and Precision@5, where other baseline models occasionally demonstrated superior performance. This suggests that the current optimization objective, primarily focused on regression accuracy, might not fully align with the goal of optimally ordering the most relevant items for a user. Future research could address this by exploring alternative or hybrid loss functions that explicitly incorporate

ranking objectives, such as listwise approaches, or by designing specific attention heads within the MHA architecture that are tailored to optimize for ranking quality.

Furthermore, while not systematically varied as an experimental factor in this study, data sparsity remains an inherent and pervasive challenge in the domain of recommender systems, especially for complex models like CA-MCGRS that aim to leverage rich contextual and multi-criteria information. In scenarios where interaction data, contextual signals, or criteria ratings are sparse, the model's ability to learn robust patterns and generalize effectively can be compromised. Addressing this could involve investigating advanced data augmentation techniques suitable for group and contextual data, or exploring transfer learning methodologies where knowledge gained from data-rich domains or related tasks could be leveraged to bolster performance in sparser target scenarios.

Looking ahead, several directions could further enhance the capabilities of CA-MCGRS. The incorporation of an even broader and more diverse spectrum of contextual dimensions warrants exploration. This could include fine-grained temporal patterns beyond simple day types or seasons, dynamic social network information reflecting evolving group structures or influences, precise user location data, or even inferred user and group emotional states, all of which could enable the model to adapt with greater agility to highly dynamic and multifaceted group scenarios.

Finally, a critical avenue for future work lies in enhancing the explainability and interpretability of CA-MCGRS. As these systems become more complex, providing users with transparent insights into how specific contextual factors (such as the impact of a Lockdown status) or particular criteria contributed to a given recommendation is paramount for fostering user trust, acceptance, and overall satisfaction [59]. Developing methods to surface these underlying decision drivers would be invaluable. Moreover, continued exploration of hybrid architectures, potentially blending the strengths of MHA-based approaches with other established paradigms like knowledge graph embeddings for richer item/context representation, or reinforcement learning for modeling sequential group decision-making processes, could lead to even more robust, versatile, and effective group recommender systems tailored for complex, real-world applications.

7 Conclusion and perspectives

In this paper, we presented and evaluated the CA-MCGRS built upon a MHA mechanism. Our approach is designed to dynamically discern and weigh the importance of diverse features arising from various contexts and multiple decision criteria, enabling the model to adapt its recommendations effectively across complex group decision-making scenarios. The core strength of CA-MCGRS lies in its MHA component, which proficiently captures the intricate, high-order relationships between group characteristics, item attributes, multi-criteria evaluations, and prevailing contextual factors.

To rigorously assess the efficacy of our CA-MCGRS, we conducted experiments across two primary aspects: firstly, by analyzing the performance impact of progressively integrating multi-criteria and multi-contextual information (Experiment 1), and secondly, by investigating the model's response to specific, single contextual factors in conjunction with multi-criteria data (Experiment 2). Our evaluations, primarily focused on RMSE and MAE, alongside ranking metrics, demonstrated that CA-MCGRS generally achieves superior predictive accuracy compared to a comprehensive suite of state-of-the-art baselines, particularly when leveraging the richness of multi-criteria and contextual information. Particularly, MHA

showed a significant improvement in accuracy when transitioning from basic GRS to MCGRS settings, and consistently led in RMSE and MAE in numerous single-context scenarios, such as the impactful Lockdown context on the ITM-Rec dataset and various temporal contexts on the MovieLens-100k dataset. While acknowledging that MHA often requires longer training times, its superior generalization capabilities, as evidenced by lower validation errors, highlight its robustness. However, we also observed that its dominance in predictive accuracy did not always translate to leading performance in top-k ranking metrics across all nuanced single-context settings.

Future research will be directed towards several key areas. Enhancing the model's capability to integrate even more complex and diverse contextual information remains a priority. Addressing the computational scalability of the MHA architecture is crucial for broader real-world applicability, perhaps through model compression techniques or more efficient attention mechanisms. Efforts will also be focused on improving the interpretability of the deep learning components within CA-MCGRS, providing users with more transparent recommendations. Furthermore, specific investigations into optimizing MHA for ranking-oriented metrics are warranted, given the observed distinctions between prediction accuracy and top-k performance. Finally, continued evaluation on larger, more varied datasets will be essential to further validate its adaptability, assess its robustness to data sparsity, and expand the applicability of CA-MCGRS, thereby advancing the frontier of intelligent group recommender systems.

Author Contributions Ngoc Luyen Le wrote the main manuscript text, developed the methodology, designed the experiments, and performed the experimental evaluations. Marie-Hélène Abel supervised the research, contributed to the theoretical framework, and provided critical revisions and feedback on the manuscript. All authors reviewed and approved the final version of the manuscript.

Data Availability No datasets were generated or analysed during the current study.

Code Availability All implementation details and source code used in this study are available on GitHub: https://github.com/lengocluyen/mhamodel.

Declarations

Ethical Approval This study does not involve any experiments on humans or animals and does not require ethical approval.

Competing interests The authors declare no competing interests.

Conflict of interest The authors declare no Conflict of interest.

References

1. Abbas, A., Zhang, L., Khan, S.U.: A survey on context-aware recommender systems based on computational intelligence techniques. Computing **97**, 667–690 (2015)
2. Adomavicius, G., Tuzhilin, A.: Context-aware recommender systems. In: Recommender systems handbook, pp. 217–253. Springer (2010)
3. Afzal, I., Yilmazel, B., Kaleli, C.: An approach for multi-context-aware multi-criteria recommender systems based on deep learning. IEEE Access (2024)
4. Al-Ghuribi, S.M., Noah, S.A.M.: Multi-criteria review-based recommender system-the state of the art. IEEE Access **7**, 169446–169468 (2019)
5. Cheng, H.T., Koc, L., Harmsen, J., Shaked, T., Chandra, et al.: Wide & deep learning for recommender systems. In: Proceedings of the 1st workshop on deep learning for recommender systems. pp. 7–10 (2016)

6. Dara, S., Chowdary, C.R., Kumar, C.: A survey on group recommender systems. J. Intell. Inf. Syst. **54**(2), 271–295 (2020)
7. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers). pp. 4171–4186 (2019)
8. Dridi, R., Tamine, L., Slimani, Y.: Exploiting context-awareness and multi-criteria decision making to improve items recommendation using a tripartite graph-based model. Inf. Process. & Manag. **59**(2), 102861 (2022)
9. Duchi, J., Hazan, E., Singer, Y.: Adaptive subgradient methods for online learning and stochastic optimization. J. Mach. Learn. Res. **12**(7) (2011)
10. Gai, K., Zhu, X., Li, H., Liu, K., Wang, Z.: Learning piece-wise linear models from large scale data for ad click prediction. arXiv:1704.05194 (2017)
11. Guo, H., Tang, R., Ye, Y., Li, Z., He, X.: Deepfm: a factorization-machine based neural network for ctr prediction. arXiv:1703.04247 (2017)
12. Hdioud, F., Frikh, B., Ouhbi, B., Khalil, I.: Multi-criteria recommender systems: A survey and a method to learn new user's profile. International Journal of Mobile Computing and Multimedia Communications (IJMCMC) **8**(4), 20–48 (2017)
13. He, X., Liao, L., Zhang, H., Nie, L., et al.: Neural collaborative filtering. In: Proceedings of the 26th international conference on world wide Web (2017)
14. Huang, T., Zhang, Z., Zhang, J.: Fibinet: combining feature importance and bilinear feature interaction for click-through rate prediction. In: Proceedings of the 13th ACM conference on recommender systems. pp. 169–177 (2019)
15. Jia, R., Zhou, X., Dong, L., Pan, S.: Hypergraph convolutional network for group recommendation. In: 2021 ieee international conference on data mining (icdm). pp. 260–269. IEEE (2021)
16. Kim, J.K., Kim, H.K., Oh, H.Y., Ryu, Y.U.: A group recommendation system for online communities. Int. J. Inf. Manag. **30**(3), 212–219 (2010)
17. van Kuijk, K., Mahmoudi, S.S., Wen, Y., Barile, F., Rienstra, T.: An argumentative framework for generating explainable group recommendations. In: Adjunct Proceedings of the 31st ACM conference on user modeling, adaptation and personalization. pp. 266–274 (2023)
18. Le, N.L., Abel, M.H., Gouspillou, P.: Towards an ontology-based recommender system for the vehicle domain. In: 3rd International conference on deep learning, artificial intelligence and robotics,(ICDLAIR). vol. 441, pp. 107–116 (2021)
19. Le, N.L., Abel, M.H., Gouspillou, P.: Improving semantic similarity measure within a recommender system based-on rdf graphs. In: International conference on information technology & systems. pp. 463–474. Springer (2023)
20. Le, N.L., Abel, M.H., Gouspillou, P.: A Personalized Recommender System Based-on Knowledge Graph Embeddings. Springer Nature Switzerland, Cham (2023)
21. Le, N.L., Abel, M.H., Gouspillou, P.: A constraint-based recommender system via rdf knowledge graphs. In: 2023 26th International conference on computer supported cooperative work in design (CSCWD). pp. 849–854 (2023)
22. Lei Ba, J., Kiros, J.R., Hinton, G.E.: Layer normalization. ArXiv e-prints pp. arXiv–1607 (2016)
23. Li, Y., Chen, H., Xu, S., Ge, Y., Tan, J., Liu, S., Zhang, Y.: Fairness in recommendation: Foundations, methods, and applications. ACM Trans. Intell. Syst. Technol. **14**(5), 1–48 (2023)
24. Lian, J., Zhou, X., Zhang, F., Chen, Z., et al.: xdeepfm: Combining explicit and implicit feature interactions for recommender systems. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. pp. 1754–1763 (2018)
25. Liao, G., Deng, X., Wan, C., Liu, X.: Group event recommendation based on graph multi-head attention network combining explicit and implicit information. Inf. Process. & Manag. **59**(2), 102797 (2022)
26. Liu, J.: Aggregating contextual information for multi-criteria online music recommendations. IEEE Access (2025)
27. Liu, Q., Yu, F., Wu, S., Wang, L.: A convolutional click prediction model. In: Proceedings of the 24th ACM international on conference on information and knowledge management. pp. 1743–1746 (2015)
28. Lu, W., Yu, Y., Chang, Y., Wang, Z., Li, C., Yuan, B.: A dual input-aware factorization machine for ctr prediction. In: Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence. pp. 3139–3145 (2021)
29. Luo, J., Li, F., Jiao, J.: A dynamic multiobjective recommendation method based on soft actor-critic with discrete actions. J. King Saud Univ. Comput. Inf. Sci. **37**(1), 1–17 (2025)

30. Luyen, L.N., Abel, M.H.: From individual to group: Developing a context-aware multi-criteria group recommender system. In: Proceeding in the 16th international conference of digital ecosystems (MEDES). Springer (2024)
31. Luyen, L.N., Abel, M.H.: Context-aware multi-criteria recommender systems using variable selection networks. In: International conference on advanced intelligent systems and informatics. pp. 3–15. Springer (2025)
32. Manouselis, N., Costopoulou, C.: Analysis and classification of multi-criteria recommender systems. World Wide Web **10**, 415–441 (2007)
33. Masthoff, J.: Group recommender systems: Combining individual models. In: Recommender systems handbook, pp. 677–702. Springer (2010)
34. Meng, X., Du, Y., Zhang, Y., Han, X.: A survey of context-aware recommender systems: from an evaluation perspective. IEEE Trans. Knowl. Data Eng. **35**(7), 6575–6594 (2022)
35. Migliorini, S., Vecchia, A.D., Belussi, A., Quintarelli, E.: Artemis: a context-aware recommendation system with crowding forecaster for the touristic domain. Inf. Syst. Front. pp. 1–27 (2024)
36. Ngoc Luyen, L., Abel, M.H., Gouspillou, P.: Enhancing context-aware recommender systems through deep feature interaction learning. J. Multi-Criteria Decis. Anal. **32**(1), e70012 (2025)
37. Niu, Z., Zhong, G., Yu, H.: A review on the attention mechanism of deep learning. Neurocomputing **452**, 48–62 (2021)
38. Nozari, R.B., Koohi, H.: A novel group recommender system based on members' influence and leader impact. Knowl.-Based Syst. **205**, 106296 (2020)
39. Paszke, A., Gross, S., Massa, F., Lerer, et al.: Pytorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems **32** (2019)
40. Qu, Y., Cai, H., Ren, K., Zhang, W., Yu, Y., Wen, Y., Wang, J.: Product-based neural networks for user response prediction. In: 2016 IEEE 16th international conference on data mining (ICDM). pp. 1149–1154. IEEE (2016)
41. Rajput, I.S., Tewari, A.S., Tiwari, A.K.: Adaptive attention mechanisms based multicriteria recommender systems for improved user preference discovery. SN Comput. Sci. **6**(4), 389 (2025)
42. Resnick, P., Varian, H.R.: Recommender systems. Commun. ACM **40**(3), 56–58 (1997)
43. Rosa, R.L., Rodriguez, D.Z., Bressan, G.: Music recommendation system based on user's sentiments extracted from social networks. IEEE Trans. Consum. Electron. **61**(3), 359–367 (2015)
44. Salau, L., Mohamed, H., Abdulsalam, Y.S., Mohammed, H.: Deep learning-based multi-criteria recommender system for technology-enhanced learning. Sci. Rep. **15**(1), 13075 (2025)
45. Shambour, Q.: A deep learning based algorithm for multi-criteria recommender systems. Knowl-based Syst **211**, 106545 (2021)
46. Shani, G., Gunawardana, A.: Evaluating recommendation systems. Recommender Systems Handbook pp. 257–297 (2011)
47. Shen, W.: Deepctr: Easy-to-use,modular and extendible package of deep-learning based ctr models. https://github.com/shenweichen/deepctr (2017)
48. Song, W., Shi, C., Xiao, Z., Duan, Z., et al.: Autoint: Automatic feature interaction learning via self-attentive neural networks. In: Proceedings of the 28th ACM international conference on information and knowledge management (2019)
49. Van Pham, H., Tuan, N.T., Tuan, L.M., Moore, P.: Idgcn: A proposed knowledge graph embedding with graph convolution network for context-aware recommendation systems. J. Organ. Comput. Electron. Commer. **35**(2), 136–156 (2025)
50. Vaswani, A.: Attention is all you need. Advances in Neural Information Processing Systems (2017)
51. Vu, S.L., Le, Q.H.: A deep learning based approach for context-aware multi-criteria recommender systems. Comput. Syst. Sci. Eng. **44**(1), 471–483 (2023)
52. Wang, R., Fu, B., Fu, G., Wang, M.: Deep & cross network for ad click predictions. In: Proceedings of the ADKDD'17, pp. 1–7 (2017)
53. Wang, Z., Chen, X.: Robust recommendation with adversarial gaussian data augmentation. In: Proceedings of the ACM Web conference 2023. pp. 897–905 (2023)
54. Xiao, J., Ye, H., He, X., Zhang, H., Wu, F., Chua, T.S.: Attentional factorization machines: Learning the weight of feature interactions via attention networks. arXiv:1708.04617 (2017)
55. Xiao, L., Min, Z., Yongfeng, Z., Zhaoquan, G., Yiqun, L., Shaoping, M.: Fairness-aware group recommendation with pareto-efficiency. In: Proceedings of the eleventh ACM conference on recommender systems. pp. 107–115 (2017)
56. Yang, Y., Xu, B., Shen, S., Shen, F., Zhao, J.: Operation-aware neural networks for user response prediction. Neural Netw. **121**, 161–168 (2020)
57. Yu, Y., Wang, Z., Yuan, B.: An input-aware factorization machine for sparse prediction. In: IJCAI. pp. 1466–1472 (2019)

58. Yuan, W., Wang, H., Yu, X., Liu, N., Li, Z.: Attention-based context-aware sequential recommendation model. Inf. Sci. **510**, 122–134 (2020)
59. Zhang, Y., Chen, X., et al.: Explainable recommendation: A survey and new perspectives. Foundations and Trends® in Information Retrieval **14**(1), 1–101 (2020)
60. Zheng, Y.: Itm-rec: An open data set for educational recommender systems. arXiv:2303.10230 (2023)
61. Zheng, Y., Shekhar, S., Jose, A.A., Rai, S.K.: Integrating context-awareness and multi-criteria decision making in educational learning. In: Proceedings of the 34th ACM/SIGAPP symposium on applied computing. pp. 2453–2460 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.